

Program no : 1**1. Introduction to SQL**

- a. Creating Tables
- b. Choosing and Describing Tables
- c. Elements of the SQL Query
- d. The System Dummy Table
- e. Selecting Tables and Columns
- f. Duplicate Information DISTINCT
- g. Sorting Information

Queries:**a. Creating Tables**

- SQL uses the `CREATE TABLE` command to define a new table in the database.
- A table consists of **columns (attributes)** and **rows (records)**.
- Example:

```
CREATE TABLE Students (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Age INT,  
    Course VARCHAR(30)  
);
```

b. Choosing and Describing Tables

- To check table structure, SQL provides the `DESCRIBE` or `DESC` command.
- Example:

```
DESC Students;
```

This shows column names, data types, and constraints.

- To see existing tables in a database:

```
SHOW TABLES;
```

c. Elements of the SQL Query

A basic SQL query has these elements:

1. **SELECT** → What data to retrieve.
2. **FROM** → The table to get data from.
3. **WHERE** → Conditions to filter data.
4. **ORDER BY** → Sort results.
5. **GROUP BY** → Group data.
6. **HAVING** → Conditions on grouped data.

```
SELECT Name, Course  
FROM Students  
WHERE Age > 18  
ORDER BY Name;
```

Typically, GROUP BY is most useful with aggregate functions (COUNT, SUM, AVG, MAX, MIN), but without them, it just gives **distinct groups** of the specified columns.

```
SELECT Course, COUNT(*) AS Total_Students  
FROM Students  
GROUP BY Course;
```

By using Having

```
SELECT Course, AVG(Age) AS Avg_Age  
FROM Students  
GROUP BY Course  
HAVING AVG(Age) > 20;
```

d. The System Dummy Table

- Some databases (like Oracle) use a **dummy table** called DUAL when you need to select a value but not from a real table.
- Example:

```
SELECT SYSDATE FROM DUAL;
```

This returns the current date/time.

- In MySQL and SQL Server, you can directly use SELECT without FROM.

e. Selecting Tables and Columns

- To fetch data:

```
SELECT * FROM Students; -- Select all columns  
SELECT Name, Course FROM Students; -- Select specific columns
```

f. Duplicate Information – DISTINCT

- By default, SELECT may return duplicates.
- Use DISTINCT to remove them.
Example:

```
SELECT DISTINCT Course FROM Students;
```

This returns unique course names only.

g. Sorting Information

- Use ORDER BY to sort results.

- ASC = ascending (default), DESC = descending.
Example:

```
SELECT Name, Age FROM Students ORDER BY Age DESC;
```

This sorts students by age in descending order.

Program no : 2

2. Data Definition Language (DDL)

a. Create b. Drop c. Alter d. Rename

a. Create

```
CREATE TABLE Students (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Age INT,  
    Course VARCHAR(30)  
);
```

b. Drop

```
DROP TABLE Students;
```

c. ALTER

1. Add a New Column

```
ALTER TABLE Students ADD Email VARCHAR(100);
```

2. Add Multiple Columns

```
ALTER TABLE Students  
ADD Address VARCHAR(200),  
ADD Phone BIGINT;
```

3. Modify Column Datatype

```
ALTER TABLE Students MODIFY Age SMALLINT;
```

Changes Age column datatype from INT to SMALLINT.

4. Change Column Name

```
ALTER TABLE Students CHANGE Name FullName VARCHAR(100);
```

5. Drop a Column

```
ALTER TABLE Students DROP COLUMN Course;
```

6. Rename Table

```
ALTER TABLE Students RENAME TO Student_Info;
```

d. RENAME

Used to **change the name** of a table (or sometimes a column, depending on the database system).

```
RENAME TABLE Students TO Student_Info;
```

Program no : 3

3. Integrity Constraints

- a. Types of Constraint
- b. Referential Integrity
- c. Defining Constraints

Types of Constraints

NOT NULL → Ensures a column cannot have NULL values.

```
CREATE TABLE Students (  
    StudentID INT NOT NULL,  
    Name VARCHAR(50) NOT NULL  
);
```

UNIQUE → Ensures all values in a column are unique (no duplicates).

```
CREATE TABLE Courses (  
    CourseID INT UNIQUE,  
    CourseName VARCHAR(50)  
);
```

PRIMARY KEY → Combination of NOT NULL + UNIQUE. Each row is uniquely identified.

```
CREATE TABLE Students (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50)  
);
```

FOREIGN KEY → Ensures a column's value matches a value in another table (relationship between tables).

```
CREATE TABLE Enrollments (  
    EnrollmentID INT PRIMARY KEY,  
    StudentID INT,  
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID)  
);
```

CHECK → Ensures a condition must be true for column values.

```
CREATE TABLE Students (  
    StudentID INT PRIMARY KEY,  
    Age INT CHECK (Age >= 18)  
);
```

DEFAULT → Assigns a default value if no value is provided.

```
CREATE TABLE Students (  
    StudentID INT PRIMARY KEY,  
    Course VARCHAR(30) DEFAULT 'B.Voc-IT'  
);
```

b. Referential Integrity:

Definition:

Referential Integrity ensures that the **relationship between two tables** remains consistent.

It means that a **foreign key in one table must match a primary key in another table**, or be NULL.

Example: Students & Enrollments

Students Table (Parent)

```
CREATE TABLE Students (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50)  
);
```

Enrollments Table (Child)

```
CREATE TABLE Enrollments (  
    EnrollmentID INT PRIMARY KEY,  
    StudentID INT,  
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE  
);
```

How Referential Integrity Works

- ✓ You can only insert a StudentID in Enrollments if that StudentID exists in Students.
- ✗ If you try:

```
INSERT INTO Enrollments VALUES (1, 99); -- 99 does not exist in Students
```

□ Error: Referential integrity violation.

- ✓ If you insert:

```
INSERT INTO Students VALUES (101, 'Akash');  
INSERT INTO Enrollments VALUES (1, 101);
```

□ Works fine (101 exists in Students).

Referential Actions

When a parent record is deleted or updated, the child table must also be handled. Options:

```
FOREIGN KEY (StudentID) REFERENCES Students(StudentID)  
ON DELETE CASCADE  
ON UPDATE CASCADE;
```

1. **ON DELETE CASCADE** → If a student is deleted, their enrollments are also deleted.
2. **ON DELETE SET NULL** → If a student is deleted, StudentID in enrollments becomes NULL.
3. **ON DELETE RESTRICT / NO ACTION** → Prevents deleting a student if enrollments exist.
4. **ON UPDATE CASCADE** → If StudentID changes in Students, it updates in Enrollments too.

Program no : 4

4. Data Manipulation Language (DML)

a. Select b. Insert c. Update d. Delete

a. SELECT → Retrieve data

Used to **fetch rows** from one or more tables.

```
SELECT * FROM Students; -- fetch all columns  
SELECT Name, Age FROM Students WHERE Age > 20;
```

b. INSERT → Add new data

-- Insert into all columns

```
INSERT INTO Students VALUES (101, 'Akash', 20, 'BCA');
```

-- Insert into selected columns

```
INSERT INTO Students (StudentID, Name, Course)  
VALUES (102, 'Priya', 'BBA');
```

c. UPDATE → Modify existing data

Used to **change values** in rows.

-- Update one column

```
UPDATE Students  
SET Age = 21  
WHERE StudentID = 101;
```

-- Update multiple columns

```
UPDATE Students  
SET Age = 22, Course = 'BCA'  
WHERE Name = 'Priya';
```

d. DELETE → Remove data

Used to **delete rows** from a table.

-- Delete one record

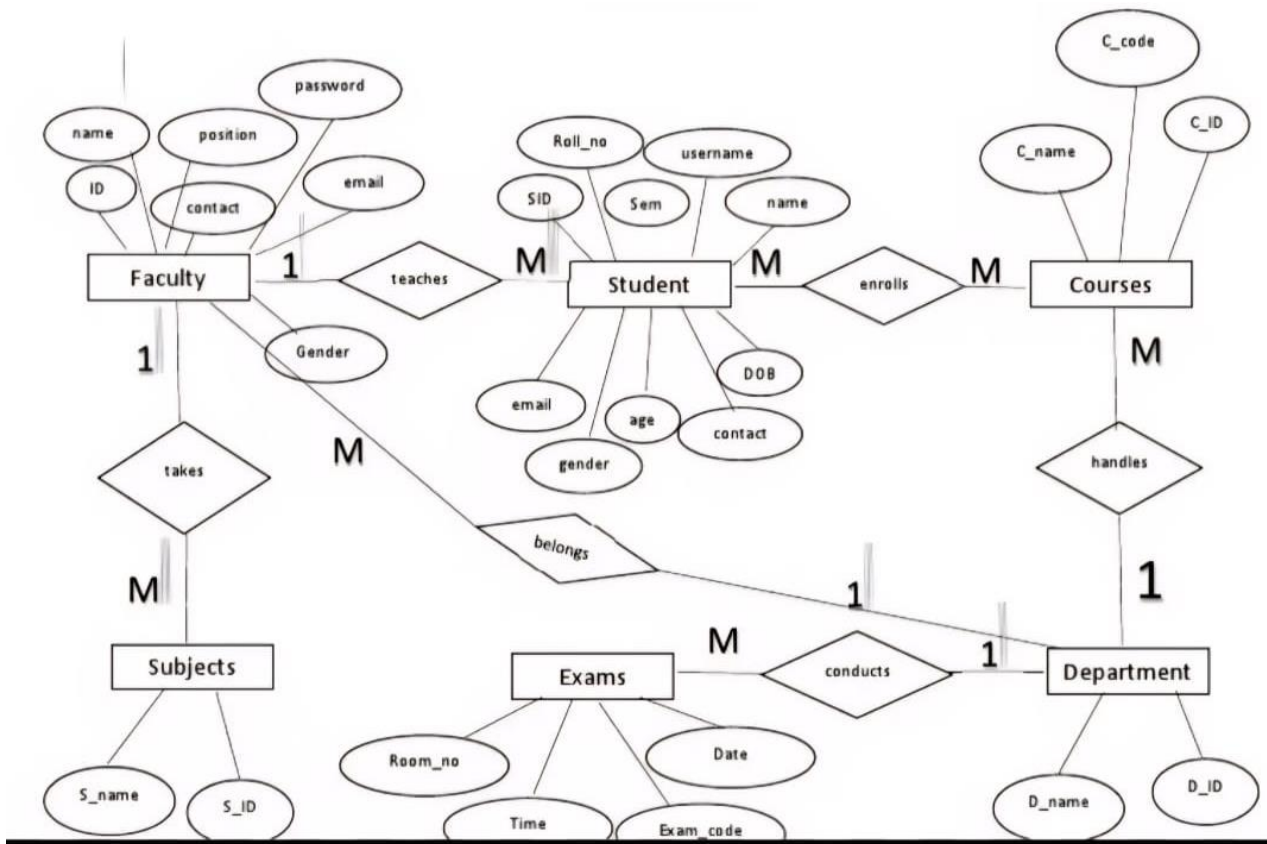
```
DELETE FROM Students WHERE StudentID = 101;
```

-- Delete all rows but keep structure

```
DELETE FROM Students;
```

Program - 5

Draw E-R diagram and convert entities and relationships to relation table for a given scenario. Two assign



conversion of entities into relational tables for a database schema.

1. Faculty Table

```
CREATE TABLE FACULTY (
  ID INT PRIMARY KEY,
  name VARCHAR(50),
  position VARCHAR(30),
  contact VARCHAR(15),
  email VARCHAR(50),
  password VARCHAR(30),
  gender VARCHAR(10)
);
```

2. Department Table

```
CREATE TABLE DEPARTMENT (
  D_ID INT PRIMARY KEY,
  D_name VARCHAR(50)
);
```

3. Student Table

```
CREATE TABLE STUDENT (
  SID INT PRIMARY KEY,
  Roll_no VARCHAR(20),
  name VARCHAR(50),
  username VARCHAR(30),
  Sem INT,
```

```
DOB DATE,  
age INT,  
gender VARCHAR(10),  
email VARCHAR(50),  
contact VARCHAR(15),  
D_ID INT,  
Faculty_ID INT,  
FOREIGN KEY (D_ID) REFERENCES DEPARTMENT(D_ID),  
FOREIGN KEY (Faculty_ID) REFERENCES FACULTY(ID)  
);
```

4. Courses Table

```
CREATE TABLE COURSES (  
  C_ID INT PRIMARY KEY,  
  C_name VARCHAR(50),  
  C_code VARCHAR(20),  
  D_ID INT,  
  FOREIGN KEY (D_ID) REFERENCES DEPARTMENT(D_ID)  
);
```

5. Subjects Table

```
CREATE TABLE SUBJECTS (  
  S_ID INT PRIMARY KEY,  
  S_name VARCHAR(50),  
  Faculty_ID INT,  
  FOREIGN KEY (Faculty_ID) REFERENCES FACULTY(ID)  
);
```

6. Exams Table

```
CREATE TABLE EXAMS (  
  Exam_code INT PRIMARY KEY,  
  Room_no VARCHAR(10),  
  Date DATE,  
  Time TIME,  
  D_ID INT,  
  FOREIGN KEY (D_ID) REFERENCES DEPARTMENT(D_ID)  
);
```

7. Enrollment Table

(Many-to-many relationship between Student and Courses)

```
CREATE TABLE ENROLLMENT (  
  SID INT,  
  C_ID INT,  
  Enrollment_Date DATE,  
  Grade CHAR(2),  
  PRIMARY KEY (SID, C_ID),  
  FOREIGN KEY (SID) REFERENCES STUDENT(SID),  
  FOREIGN KEY (C_ID) REFERENCES COURSES(C_ID)  
);
```

Relationships

Relationship	Type	Foreign Keys
Faculty teaches Student	1 : M	Faculty.ID → Student.Faculty_ID
Student enrolls Courses	M : M	Resolved by Enrollment table
Faculty takes Subjects	1 : M	Faculty.ID → Subjects.Faculty_ID
Department handles Courses	1 : M	Department.D_ID → Courses.D_ID
Department conducts Exams	1 : M	Department.D_ID → Exams.D_ID
Student belongs Department	M : 1	Department.D_ID → Student.D_ID

Program -9

Group By

- Group Function Examples
- Group Function with Having

Table Creation:

```
CREATE TABLE Employee (
  emp_id INT PRIMARY KEY,
  emp_name VARCHAR(50),
  department VARCHAR(30),
  salary DECIMAL(10,2)
);
```

Insert Sample Data:

```
INSERT INTO Employee VALUES
(1, 'Ravi', 'HR', 30000),
(2, 'Priya', 'IT', 45000),
(3, 'Kiran', 'IT', 55000),
(4, 'Manoj', 'Accounts', 25000),
(5, 'Divya', 'HR', 40000),
(6, 'Suresh', 'Accounts', 30000),
(7, 'Meena', 'IT', 60000);
```

a. Group Function Example:

Find the total salary paid for each department.

```
SELECT department, SUM(salary) AS Total_Salary
FROM Employee
GROUP BY department;
```

Output:

<u>Department</u>	<u>Total_Salary</u>
HR	70000.00
IT	160000.00
Accounts	55000.00

b. Group Function with HAVING Clause:

Display departments whose total salary exceeds 60,000.

```
SELECT department, SUM(salary) AS Total_Salary  
FROM Employee  
GROUP BY department  
HAVING SUM(salary) > 60000;
```

Output:

<u>Department</u>	<u>Total_Salary</u>
HR	70000.00
IT	160000.00

Explanation:

GROUP BY groups the rows based on the department column.

SUM(salary) calculates the total salary for each department.

HAVING works like a WHERE clause but is used with aggregate functions.

BMSCW

Program -12

Views

- a. Create View
- b. With Check Option

Table Creation

```
CREATE TABLE Employee (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    department VARCHAR(30),  
    salary DECIMAL(10,2)  
);
```

Insert Sample Data

```
INSERT INTO Employee VALUES  
(101, 'Ravi', 'HR', 30000),  
(102, 'Priya', 'IT', 45000),  
(103, 'Kiran', 'IT', 55000),  
(104, 'Divya', 'Accounts', 25000),  
(105, 'Meena', 'HR', 40000);
```

a. Create View

A view is a virtual table created from one or more existing tables.

Create a view that shows only IT department employees:

```
CREATE VIEW IT_Employees AS  
SELECT emp_id, emp_name, salary  
FROM Employee  
WHERE department = 'IT';
```

View the Data:

```
SELECT * FROM IT_Employees;
```

Output:

<u>emp_id</u>	<u>emp_name</u>	<u>salary</u>
102	Priya	45000.00
103	Kiran	55000.00

b. Create View with CHECK OPTION

The WITH CHECK OPTION ensures that only rows satisfying the view condition can be inserted or updated through the view.

```
CREATE VIEW HR_Employees AS  
SELECT emp_id, emp_name, salary, department  
FROM Employee  
WHERE department = 'HR'
```

Trying to Insert Through View (Valid Case)

```
INSERT INTO HR_Employees VALUES (106, 'Anitha', 38000, 'HR');
```

Successfully inserted because it satisfies department = 'HR'.

Trying to Insert Through View (Invalid Case)

```
INSERT INTO HR_Employees VALUES (107, 'Vijay', 50000, 'IT');
```

Error:

ERROR: CHECK OPTION failed — cannot insert row because department != 'HR'

Explanation:

View

A virtual table based on the result of a SELECT query.

WITH CHECK OPTION

Prevents changes that violate the view's condition.

Purpose Provides data security, simplification, and controlled access.

Program -15

Transaction Control Language

- a. Commit
- b. Rollback
- c. Savepoint
- d. Set Transaction

Create the Table

```
CREATE TABLE student (  
    student_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    course VARCHAR(30),  
    marks INT  
);
```

Insert Some Records (Uncommitted yet)

```
INSERT INTO student VALUES (1, 'Meghana', 'BCA', 85);  
INSERT INTO student VALUES (2, 'Raj', 'BSc', 78);  
INSERT INTO student VALUES (3, 'Meena', 'BCom', 90);
```

```
SELECT * FROM student;
```

Save Changes Permanently using COMMIT

```
COMMIT;
```

✓ These 3 student records are now **permanently stored** in the database.

Using SAVEPOINT and ROLLBACK

Start a new transaction

```
INSERT INTO student VALUES (4, 'Kiran', 'BCA', 88);
```

```
SAVEPOINT sp1;
```

```
INSERT INTO student VALUES (5, 'Divya', 'BSc', 92);
```

```
SAVEPOINT sp2;
```

```
INSERT INTO student VALUES (6, 'Manu', 'BCom', 80);
```

-- Suppose you want to undo only the last insert

```
ROLLBACK TO sp2;
```

✓ Records 1–5 will remain.

✗ Record for *Manu* (student_id = 6) is undone.

Now you can make the remaining data permanent:

```
COMMIT;
```

Step 6: Setting Transaction Properties

You can control how a transaction behaves using **SET TRANSACTION**.

(a) Read-only transaction

```
SET TRANSACTION READ ONLY;
```

```
SELECT * FROM student;
```

```
COMMIT;
```

✓ You can only **view** the data — not insert or update.

(b) Read-write transaction

```
SET TRANSACTION READ WRITE;
```

```
UPDATE student  
SET marks = marks + 5  
WHERE course = 'BCA';
```

```
COMMIT;
```

✓ You can modify data and save it.

Final Result (after all TCL operations)

```
SELECT * FROM student;
```

student_id	name	course	marks
1	Meghana	BCA	85
2	Raj	BSc	78
3	Meena	BCom	90
4	Kiran	BCA	93
5	Divya	BSc	92

<u>TCL Command</u>	<u>Purpose</u>	<u>Example Used</u>	<u>Result</u>
COMMIT	Save all changes permanently	COMMIT ;	Data stored permanently
ROLLBACK	Undo all uncommitted changes	ROLLBACK ;	Changes reverted
SAVEPOINT	Create points for partial rollback	SAVEPOINT sp1 ; ROLLBACK TO sp2 ;	Undo partial data
SET TRANSACTION	Define transaction behavior	SET TRANSACTION READ ONLY ;	Read-only or read-write control

BMSCW